

Events & Event Handling — Events এবং User Interaction

এখন আমরা JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ practical বিষয়গুলোর একটি শিখব — **Events**! DOM Chapter-এ আমরা শিখেছি কীভাবে HTML element select করে পরিবর্তন করতে হয়। কিন্তু একটি webpage-এ user যখন বাটন ক্লিক করে, ইনপুটে লিখে, মাউস সরায়, কিবোর্ড প্রেস করে কিংবা ফর্ম সাবমিট করে—তখন জাভাস্ক্রিপ্ট কীভাবে বুঝবে যে কিছু একটা ঘটেছে? এর উত্তর হলো 🙌 **Event**!

User / Browser Action → Event Triggered → Event Listener (Callback) → Event Object → Dynamic Response

ইভেন্ট হ্যান্ডলিং হলো ইন্টারঅ্যাক্টিভ ও ডায়নামিক ফ্রন্টএন্ড ওয়েব ডেভেলপমেন্টের চালিকাশক্তি! 🔥

PART 01 — EVENT FOUNDATIONS, HANDLERS & ADDEVENTLISTENER (16.1 – 16.10)

16.1 Event কী?

Event হলো webpage-এ ঘটে যাওয়া কোনো action বা ঘটনা।

সহজভাবে:

User বা browser-এর কোনো action ঘটলে সেটাকে Event বলা হয়।

💡 REAL-LIFE EXAMPLE

ধরো একটি door:

```
Door
↓
Someone knocks
↓
You hear it
↓
You open the door
```

Webpage:

```
Button
↓
User clicks
↓
Browser detects event
↓
JavaScript runs code
```

16.2 Common Events

JavaScript-এ অনেক ধরনের event আছে।

Event	Meaning
<code>click</code>	Mouse click
<code>dblclick</code>	Double click
<code>mousedown</code>	Mouse button press
<code>mouseup</code>	Mouse button release
<code>mousemove</code>	Mouse move
<code>mouseenter</code>	Element-এর ভিতরে mouse ঢোকে
<code>mouseleave</code>	Element থেকে mouse বের হয়
<code>keydown</code>	Keyboard key press
<code>keyup</code>	Keyboard key release
<code>input</code>	Input value change
<code>change</code>	Form value change/commit
<code>submit</code>	Form submit
<code>focus</code>	Element focus পায়
<code>blur</code>	Element focus হারায়
<code>load</code>	Resource/page load
<code>scroll</code>	Page scroll
<code>resize</code>	Window resize

16.3 Event Handler কী?

Event ঘটলে যে function execute হয় তাকে সাধারণভাবে **Event Handler** বলা হয়।

Example:

16.3 JAVASCRIPT

RUN

```
function handleClick() {  
  console.log("Button clicked!");  
}
```

এই function-কে click event-এর সাথে connect করলে এটি event handler হিসেবে কাজ করবে।

16.4 প্রথম Event Example

HTML:

16.4 HTML

MARKUP

```
<button id="btn">Click Me</button>
```

JavaScript:

16.4 JAVASCRIPT

RUN

```
let button = document.getElementById("btn");  
  
button.addEventListener("click", function () {  
  console.log("Button clicked!");  
});
```

Button click করলে:

```
Button clicked!
```

16.5 addEventListener()

এটি JavaScript-এ event handle করার সবচেয়ে গুরুত্বপূর্ণ method।

Syntax:

```
16.5 JAVASCRIPT RUN
element.addEventListener("event", function () {
  // code
});
```

Example:

```
16.5 JAVASCRIPT RUN
button.addEventListener("click", function () {
  console.log("Hello");
});
```

এখানে:

```
button
  ↓
addEventListener()
  ↓
"click"
  ↓
function()
```

16.6 Named Function ব্যবহার

Anonymous function-এর পরিবর্তে named function-ও ব্যবহার করা যায়।

```
16.6 JAVASCRIPT RUN
function handleClick() {
  console.log("Clicked!");
}

button.addEventListener("click", handleClick);
```

★ খুব গুরুত্বপূর্ণ

সঠিক:

```
16.6 JAVASCRIPT RUN
button.addEventListener("click", handleClick);
```

ভুল:

```
16.6 JAVASCRIPT RUN
button.addEventListener("click", handleClick());
```

কারণ `handleClick()` দিলে function তখনই execute হয়ে যাবে।

16.7 onclick` Property

Event handle করার আরেকটি উপায়:

```
16.7 JAVASCRIPT RUN
button.onclick = function () {
  console.log("Clicked!");
};
```

এটি কাজ করে।

কিন্তু সাধারণত modern code-এ:

```
16.7 JAVASCRIPT RUN
addEventListener()
```

বেশি flexible।

16.8 HTML Inline Event

HTML-এর ভিতরেও event লেখা যায়:

16.8 HTML

MARKUP

```
<button onclick="sayHello()">
  Click Me
</button>
```

JavaScript:

16.8 JAVASCRIPT

RUN

```
function sayHello() {
  alert("Hello!");
}
```

এটি কাজ করলেও বড় application-এ HTML এবং JavaScript আলাদা রাখা সাধারণত cleaner approach।

16.9 addEventListener() কেন গুরুত্বপূর্ণ?

একই element-এ multiple event listener দেওয়া যায়।

16.9 JAVASCRIPT

RUN

```
button.addEventListener("click", function () {
  console.log("First");
});

button.addEventListener("click", function () {
  console.log("Second");
});
```

Click করলে:

TEXT

First
Second

16.10 Event Listener Remove

একটি listener remove করতে একই function reference দরকার।

16.10 JAVASCRIPT

RUN

```
function handleClick() {
  console.log("Clicked");
}

button.addEventListener("click", handleClick);

button.removeEventListener("click", handleClick);
```

এখন click করলে function আর execute হবে না।

★ IMPORTANT

এটি কাজ করবে না:

16.10 JAVASCRIPT

RUN

```
button.addEventListener("click", function () {
  console.log("Clicked");
});

button.removeEventListener("click", function () {
  console.log("Clicked");
});
```

কারণ দুইটি anonymous function আলাদা function object।

16.11 Event Object

Event ঘটলে browser একটি **event object** তৈরি করে।

16.11 JAVASCRIPT

RUN

```
button.addEventListener("click", function (event) {
  console.log(event);
});
```

এখানে:

16.11 JAVASCRIPT

RUN

```
event
```

এর মধ্যে event-এর অনেক information থাকে।

16.12 event.target

যে element-এ event ঘটেছে সেটি:

16.12 JAVASCRIPT

RUN

```
event.target
```

Example:

16.12 JAVASCRIPT

RUN

```
button.addEventListener("click", function (event) {
  console.log(event.target);
});
```

16.12 HTML

MARKUP

```
<button id="btn">Click Me</button>
```

16.13 event.currentTarget

যে element-এর ওপর listener attached হয়েছে:

16.13 JAVASCRIPT

RUN

```
event.currentTarget
```

অনেক সময়:

```
target === currentTarget
```

হতে পারে।

কিন্তু event bubbling-এর সময় এদের মধ্যে পার্থক্য খুব গুরুত্বপূর্ণ হয়ে যায়।

16.14 event.type

Event-এর নাম পাওয়া যায়:

16.14 JAVASCRIPT

RUN

```
button.addEventListener("click", function (event) {
  console.log(event.type);
});
```

TEXT

```
click
```

16.15 event.timeStamp

Event কখন ঘটেছে তার timing information:

16.15 JAVASCRIPT

RUN

```
button.addEventListener("click", function (event) {  
  console.log(event.timeStamp);  
});
```

PART 03 — MOUSE & KEYBOARD INTERACTION EVENTS (16.16 – 16.29)

16.16 Mouse Click Event

16.16 JAVASCRIPT

RUN

```
button.addEventListener("click", function () {  
  console.log("Clicked");  
});
```

16.17 Double Click

16.17 JAVASCRIPT

RUN

```
button.addEventListener("dblclick", function () {  
  console.log("Double clicked");  
});
```

16.18 Mouse Down

Mouse button press করলে:

16.18 JAVASCRIPT

RUN

```
button.addEventListener("mousedown", function () {  
  console.log("Mouse button pressed");  
});
```

16.19 Mouse Up

Mouse button release:

16.19 JAVASCRIPT

RUN

```
button.addEventListener("mouseup", function () {  
  console.log("Mouse button released");  
});
```

16.20 Mouse Move

16.20 JAVASCRIPT

RUN

```
document.addEventListener("mousemove", function () {  
  console.log("Mouse moving");  
});
```

⚠ এটি খুব ঘন ঘন fire হতে পারে। তাই performance-sensitive code-এ সাবধান হতে হবে।

16.21 mouseenter

Mouse কোনো element-এর ভিতরে ঢুকলে:

16.21 JAVASCRIPT

RUN

```
box.addEventListener("mouseenter", function () {  
  console.log("Mouse entered");  
});
```

16.22 mouseleave

Mouse element থেকে বের হলে:

16.22 JAVASCRIPT

RUN

```
box.addEventListener("mouseleave", function () {  
  console.log("Mouse left");  
});
```

16.23 Keyboard Events

Keyboard-এর জন্য গুরুত্বপূর্ণ:

TEXT

keydown
keyup

16.24 keydown

কোনো key press করলে:

16.24 JAVASCRIPT

RUN

```
document.addEventListener("keydown", function (event) {  
  console.log(event.key);  
});
```

তুমি **A** press করলে:

TEXT

a

Enter:

TEXT

Enter

Escape:

TEXT

Escape

16.25 keyup

Key release করলে:

16.25 JAVASCRIPT

RUN

```
document.addEventListener("keyup", function (event) {  
  console.log(event.key);  
});
```

16.26 event.code

`event.key` key-এর value দেয়।

`event.code` physical keyboard key-এর নাম দেয়।

16.26 JAVASCRIPT

RUN

```
document.addEventListener("keydown", function (event) {  
  
    console.log("Key:", event.key);  
    console.log("Code:", event.code);  
  
});
```

A press করলে browser সাধারণত:

TEXT

Key: a
Code: KeyA

16.27 event.key বনাম event.code

key	code
Press করা key-এর value	Physical key
"a"	"KeyA"
"Enter"	"Enter"
" "	"Space"

16.28 Keyboard Shortcut

ধরো:

TEXT

Ctrl + S

detect করতে:

16.28 JAVASCRIPT

RUN

```
document.addEventListener("keydown", function (event) {  
  
    if (event.ctrlKey && event.key === "s") {  
        console.log("Save shortcut");  
    }  
  
});
```

16.29 Modifier Keys

Event object-এ পাওয়া যায়:

```
16.29 JAVASCRIPT RUN
event.ctrlKey
event.shiftKey
event.altKey
event.metaKey
```

Example:

```
16.29 JAVASCRIPT RUN
document.addEventListener("keydown", function (event) {

    if (event.shiftKey) {
        console.log("Shift is pressed");
    }

});
```

PART 04 — FORM, INPUT, FOCUS & WINDOW EVENTS (16.30 – 16.42)

16.30 Input Event

User input field-এ typing করলে `input` event fire হয়।

HTML:

```
16.30 HTML MARKUP
<input id="nameInput">
```

JavaScript:

```
16.30 JAVASCRIPT RUN
let input = document.getElementById("nameInput");

input.addEventListener("input", function (event) {

    console.log(event.target.value);

});
```

User লিখলে:

```
TEXT
S
Sh
Sha
Shar
Shariar
```

প্রতিবার event fire হতে পারে।

16.31 change`Event

```
16.31 JAVASCRIPT RUN
input.addEventListener("change", function () {
    console.log("Value changed");
});
```

`input` এবং `change` এক নয়।

সাধারণভাবে:

```
input
→ typing/change-এর সময় continuously fire

change
→ value committed/changed হওয়ার পর fire
```

16.32 Focus Event

Input focus পেল:

16.32 JAVASCRIPT

RUN

```
input.addEventListener("focus", function () {  
    console.log("Input focused");  
});
```

16.33 Blur Event

Input focus হারালে:

16.33 JAVASCRIPT

RUN

```
input.addEventListener("blur", function () {  
    console.log("Input lost focus");  
});
```

16.34 Form Submit Event

HTML:

16.34 HTML

MARKUP

```
<form id="loginForm">  
    <input id="email">  
  
    <button type="submit">  
        Login  
    </button>  
  
</form>
```

JavaScript:

16.34 JAVASCRIPT

RUN

```
let form = document.getElementById("loginForm");  
  
form.addEventListener("submit", function () {  
    console.log("Form submitted");  
});
```

16.35 preventDefault()

Form submit করলে browser সাধারণত default action করতে পারে, যেমন page reload/navigation।

এটা বন্ধ করতে:

16.35 JAVASCRIPT

RUN

```
form.addEventListener("submit", function (event) {  
    event.preventDefault();  
  
    console.log("Submit handled by JavaScript");  
});
```

এটি web development-এ অত্যন্ত গুরুত্বপূর্ণ।

16.36 Real-Life Login Form

16.36 JAVASCRIPT

RUN

```
form.addEventListener("submit", function (event) {

    event.preventDefault();

    let email = document.getElementById("email").value;

    if (email === "") {
        console.log("Email is required");
        return;
    }

    console.log("Login request sent");

});
```

16.37 Checkbox Change

16.37 HTML

MARKUP

```
<input id="terms" type="checkbox">
```

16.37 JAVASCRIPT

RUN

```
let terms = document.getElementById("terms");

terms.addEventListener("change", function () {

    console.log(terms.checked);

});
```

16.38 Select Change

HTML:

16.38 HTML

MARKUP

```
<select id="department">

    <option value="cse">CSE</option>
    <option value="eee">EEE</option>
    <option value="bba">BBA</option>

</select>
```

JavaScript:

16.38 JAVASCRIPT

RUN

```
department.addEventListener("change", function () {

    console.log(department.value);

});
```

16.39 Window Events

সমস্ত HTML elements নয়, `window` -এরও events আছে।

16.39 JAVASCRIPT

RUN

```
window.addEventListener("resize", function () {

    console.log("Window resized");

});
```

16.40 Scroll Event

16.40 JAVASCRIPT

RUN

```
window.addEventListener("scroll", function () {  
    console.log("Page scrolling");  
});
```

⚠ Scroll event-ও অনেকবার fire হতে পারে।

16.41 Load Event

Page/resources load হলে:

16.41 JAVASCRIPT

RUN

```
window.addEventListener("load", function () {  
    console.log("Page fully loaded");  
});
```

16.42 DOMContentLoaded

HTML document parse শেষ হলে:

16.42 JAVASCRIPT

RUN

```
document.addEventListener("DOMContentLoaded", function () {  
  
    console.log("DOM ready");  
});
```

এটি DOM manipulation শুরু করার জন্য খুব usefull

16.43 Event Bubbling

এটি Chapter 16-এর সবচেয়ে গুরুত্বপূর্ণ advanced concept-এর একটি।

HTML:

16.43 HTML

MARKUP

```
<div id="parent">
  <button id="child">
    Click
  </button>
</div>
```

JavaScript:

16.43 JAVASCRIPT

RUN

```
let parent = document.getElementById("parent");
let child = document.getElementById("child");

child.addEventListener("click", function () {
  console.log("Child");
});

parent.addEventListener("click", function () {
  console.log("Parent");
});
```

Button click করলে:

```
Child
Parent
```

কেন?

কারণ event child থেকে parent-এর দিকে **bubble** করে।

16.44 Event Bubbling-এর Mental Model

```
Button
  ↑
Parent
  ↑
Body
  ↑
HTML
  ↑
Document
  ↑
Window
```

Event সাধারণত target থেকে উপরের দিকে propagate করে।

16.45 event.target বনাম event.currentTarget

এখন পার্থক্যটা বুঝা।

16.45 JAVASCRIPT

RUN

```
parent.addEventListener("click", function (event) {  
  
    console.log("Target:", event.target);  
    console.log("Current:", event.currentTarget);  
  
});
```

যদি child button click করো:

```
Target  
→ button  
  
CurrentTarget  
→ parent
```

★ মনে রাখো

```
target  
→ যেখানে event শুরু হয়েছে  
  
currentTarget  
→ যেখানে listener attached
```

16.46 stopPropagation()

Event bubbling বন্ধ করতে:

16.46 JAVASCRIPT

RUN

```
child.addEventListener("click", function (event) {  
  
    event.stopPropagation();  
  
    console.log("Child clicked");  
  
});
```

এখন parent listener সাধারণ bubbling-এর কারণে execute হবে না।

16.47 Event Capturing

Event propagation-এর একটি phase হলো **capturing**।

সাধারণভাবে event flow:

```
Capturing  
↓  
Target  
↓  
Bubbling
```

16.48 Capturing Listener

`addEventListener()` -এর third argument বা options ব্যবহার করে capture করা যায়।

16.48 JAVASCRIPT

RUN

```
parent.addEventListener(
  "click",
  function () {
    console.log("Parent capture");
  },
  true
);
```

এখন parent capture phase-এ event পেতে পারে।

Modern style:

16.48 JAVASCRIPT

RUN

```
parent.addEventListener(
  "click",
  function () {
    console.log("Parent capture");
  },
  { capture: true }
);
```

16.49 Event Propagation

পুরো concept:

```
Window
↓
Document
↓
HTML
↓
Body
↓
Parent
↓
Button
↑
Target
↑
Parent
↑
Body
↑
Document
↑
Window
```

প্রথম অংশ:

Capturing

তারপর:

Target

শেষে:

Bubbling

16.50 Event Delegation

এটি খুব গুরুত্বপূর্ণ professional technique।

ধরো:

16.50 HTML

MARKUP

```
<ul id="list">

  <li>Apple</li>
  <li>Banana</li>
  <li>Mango</li>

</ul>
```

প্রতিটি `<li>` -তে আলাদা listener না দিয়ে parent-এ listener দিতে পারো।

16.50 JAVASCRIPT

RUN

```
let list = document.getElementById("list");

list.addEventListener("click", function (event) {

  if (event.target.matches("li")) {
    console.log(event.target.textContent);
  }

});
```

Apple click:

TEXT

Apple

Mango click:

TEXT

Mango

16.51 Event Delegation কেন দরকার?

ধরো 1000টি button আছে।

খারাপ approach:

```
Button 1 → listener
Button 2 → listener
Button 3 → listener
...
Button 1000 → listener
```

Delegation:

```
Parent
↓
একটি listener
↓
Target identify
```

এতে code cleaner এবং অনেক ক্ষেত্রে resource usage কমে।

16.52 Dynamic Elements এবং Event Delegation

এটি খুব গুরুত্বপূর্ণ।

ধরো পরে JavaScript দিয়ে নতুন `<li>` তৈরি করলে:

16.52 JAVASCRIPT

RUN

```
let li = document.createElement("li");

li.textContent = "Orange";

list.append(li);
```

Parent-এর delegated listener থাকলে নতুন `li` -তেও click handling কাজ করতে পারে।

16.53 preventDefault() বনাম stopPropagation()

দুটো একদম আলাদা।

Method	কাজ
<code>preventDefault()</code>	Browser-এর default action বন্ধ
<code>stopPropagation()</code>	Event propagation বন্ধ

Example:

16.53 JAVASCRIPT

RUN

```
event.preventDefault();
```

→ form submit/navigation-এর default behavior বন্ধ করতে পারে।

16.53 JAVASCRIPT

RUN

```
event.stopPropagation();
```

→ parent-এর দিকে event propagation বন্ধ করে।

16.54 stopImmediatePropagation()

একই element-এ multiple listener থাকলে অন্য listener-গুলোর execution-ও থামাতে পারে।

16.54 JAVASCRIPT

RUN

```
event.stopImmediatePropagation();
```

এটি advanced use case-এ প্রয়োজন হয়।

16.55 Event Options

`addEventListener()` -এর third parameter-এ options দেওয়া যায়।

16.55 JAVASCRIPT

RUN

```
button.addEventListener(
  "click",
  handleClick,
  {
    once: true
  }
);
```

16.56 once

16.56 JAVASCRIPT

RUN

```
button.addEventListener(  
  "click",  
  function () {  
    console.log("Runs once");  
  },  
  {  
    once: true  
  }  
);
```

Button যতবার click করো, শুধু প্রথমবার execute হবে।

16.57 capture

16.57 JAVASCRIPT

RUN

```
element.addEventListener(  
  "click",  
  handleClick,  
  {  
    capture: true  
  }  
);
```

Capture phase-এ listener চালাবে।

16.58 passive

16.58 JAVASCRIPT

RUN

```
window.addEventListener(  
  "scroll",  
  handleScroll,  
  {  
    passive: true  
  }  
);
```

Browser-কে জানানো হয় যে listener সাধারণত `preventDefault()` ব্যবহার করবে না।

বিশেষ করে touch/scroll performance-এর ক্ষেত্রে এটি গুরুত্বপূর্ণ হতে পারে।

16.59 Real-Life Example — Character Counter

HTML:

16.59 HTML

MARKUP

```
<textarea id="message"></textarea>

<p>
  Characters:
  <span id="count">0</span>
</p>
```

JavaScript:

16.59 JAVASCRIPT

RUN

```
let message = document.getElementById("message");
let count = document.getElementById("count");

message.addEventListener("input", function () {

  count.textContent = message.value.length;

});
```

User লিখলে:

TEXT

Hello

দেখাবে:

TEXT

Characters: 5

16.60 Real-Life Example — Live Search

HTML:

16.60 HTML

MARKUP

```
<input id="search" placeholder="Search ... ">

<ul id="results"></ul>
```

JavaScript:

16.60 JAVASCRIPT

RUN

```
let search = document.getElementById("search");
let results = document.getElementById("results");

let products = [
  "Laptop",
  "Mouse",
  "Keyboard",
  "Monitor"
];

search.addEventListener("input", function () {

  let keyword = search.value.toLowerCase();

  results.innerHTML = "";

  let filtered = products.filter(function (product) {
    return product.toLowerCase().includes(keyword);
  });

  filtered.forEach(function (product) {

    let li = document.createElement("li");

    li.textContent = product;

    results.append(li);

  });

});
```

প্রক্রিয়া:

```
User types
↓
input event
↓
Search data
↓
filter()
↓
DOM update
```

16.61 Real-Life Example — Password Visibility

HTML:

16.61 HTML

MARKUP

```
<input id="password" type="password">

<button id="toggle">
  Show
</button>
```

JavaScript:

16.61 JAVASCRIPT

RUN

```
let password = document.getElementById("password");
let toggle = document.getElementById("toggle");

toggle.addEventListener("click", function () {

  if (password.type === "password") {

    password.type = "text";
    toggle.textContent = "Hide";

  } else {

    password.type = "password";
    toggle.textContent = "Show";

  }

});
```

এটি real-world login/signup form-এ ব্যবহৃত হয়।

16.62 Real-Life Example — Form Validation

16.62 JAVASCRIPT

RUN

```
form.addEventListener("submit", function (event) {

  event.preventDefault();

  let email = document.getElementById("email").value;

  if (!email.includes("@")) {

    console.log("Invalid email");

    return;

  }

  console.log("Form submitted successfully");

});
```

16.63 Event Handler-এর ভিতরে `this`

Regular function ব্যবহার করলে event listener-এর ক্ষেত্রে `this` সাধারণত যে element-এ listener attached হয়েছে সেটিকে refer করতে পারে।

16.63 JAVASCRIPT

RUN

```
button.addEventListener("click", function () {  
  
    console.log(this);  
  
});
```

সাধারণত:

`this → button`

কিন্তু arrow function-এর নিজের `this` binding নেই।

16.63 JAVASCRIPT

RUN

```
button.addEventListener("click", () => {  
  
    console.log(this);  
  
});
```

এখানে `this` lexical context থেকে আসবে।

16.64 Event Listener + Arrow Function

Arrow function অবশ্যই ব্যবহার করা যায়:

16.64 JAVASCRIPT

RUN

```
button.addEventListener("click", (event) => {  
  
    console.log(event.target);  
  
});
```

এক্ষেত্রে element access করতে:

16.64 JAVASCRIPT

RUN

`event.currentTarget`

ব্যবহার করা সুবিধাজনক হতে পারে।

16.65 Event Flow-এর Complete Example

HTML:

16.65 HTML

MARKUP

```
<div id="parent">

  <button id="button">
    Click Me
  </button>

</div>
```

JavaScript:

16.65 JAVASCRIPT

RUN

```
let parent = document.getElementById("parent");
let button = document.getElementById("button");

parent.addEventListener("click", function (event) {

  console.log(
    "Parent:",
    event.target,
    event.currentTarget
  );

});

button.addEventListener("click", function (event) {

  console.log(
    "Button:",
    event.target,
    event.currentTarget
  );

});
```

Button click করলে:

```
Button: button button
Parent: button parent
```

এটাই `target` বনাম `currentTarget` বোঝার সবচেয়ে ভালো example।

16.66 Must Know

Chapter 16 থেকে এগুলো অবশ্যই জানতে হবে:

Basic

16.66 JAVASCRIPT

RUN

```
addEventListener()
removeEventListener()
```

Mouse

TEXT

```
click
dblclick
mousedown
mouseup
mousemove
mouseenter
mouseleave
```

Keyboard

TEXT

```
keydown
keyup
```

Form

TEXT

```
input
change
focus
blur
submit
```

Browser

TEXT

```
load
DOMContentLoaded
scroll
resize
```

Event Object

16.66 JAVASCRIPT

RUN

```
event.target
event.currentTarget
event.type
event.key
event.code
event.ctrlKey
event.shiftKey
event.altKey
event.metaKey
```

Event Control

16.66 JAVASCRIPT

RUN

```
preventDefault()
stopPropagation()
stopImmediatePropagation()
```

Event Propagation

```
Capturing
↓
Target
↓
Bubbling
```

Event Delegation

16.66 JAVASCRIPT

RUN

```
parent.addEventListener( ... )
```

CHEAT SHEET

🗨 Quick Cheat Sheet — Common Events & Listeners

JAVASCRIPT

RUN

```
// Click
button.addEventListener("click", handler);

// Keyboard
document.addEventListener("keydown", handler);

// Input
input.addEventListener("input", handler);

// Form
form.addEventListener("submit", handler);

// Stop default
event.preventDefault();

// Stop bubbling
event.stopPropagation();

// Target
event.target;

// Listener element
event.currentTarget;

// Keyboard
event.key;
event.code;

// Modifier
event.ctrlKey;
event.shiftKey;
event.altKey;
event.metaKey;
```

LEVEL: BEGINNER

1. একটি button click করলে:

```
TEXT
```

```
Hello JavaScript
```

console-এ দেখাও।

2. একটি button click করলে তার text পরিবর্তন করো।

3. Mouse enter করলে box-এর color পরিবর্তন করো।

4. Keyboard-এ যেকোনো key press করলে key-এর নাম console-এ দেখাও।

LEVEL: INTERMEDIATE

5. একটি input-এর value live `<p>` element-এ দেখাও।

6. একটি textarea-এর character counter তৈরি করো।

7. একটি form submit করলে page reload বন্ধ করো।

8. একটি checkbox checked কিনা তা `change` event দিয়ে detect করো।

LEVEL: ADVANCED

9. `Ctrl + S` shortcut detect করো এবং browser-এর default behavior prevent করো।

10. Event bubbling demonstrate করো:

```
Parent
  ↓
Child
```

11. `stopPropagation()` ব্যবহার করে bubbling বন্ধ করো।

12. Event delegation ব্যবহার করে একটি dynamic Todo List বানাও।

13. Parent-এর একটি listener ব্যবহার করে তার সব child button-এর click handle করো।

SUMMARY **Chapter 16 Final Mental Map — Event Flow & Propagation**

পুরো chapter-টা এই flow-তে মনে রাখো:

```
User Action
  ↓
Event
  ↓
Event Listener
  ↓
Handler Function
  ↓
Event Object
  ↓
JavaScript Logic
  ↓
DOM Update
  ↓
UI Changes
```

আর advanced event system:

```
Event
  ↓
Capturing
  ↓
Target
  ↓
Bubbling
  ↓
Event Delegation
```

★ সবচেয়ে গুরুত্বপূর্ণ পাঠ্য

```
preventDefault()
→ Browser-এর default কাজ বন্ধ করে

stopPropagation()
→ Event-এর propagation বন্ধ করে

target
→ যেখানে event ঘটেছে

currentTarget
→ যেখানে listener attached

input
→ user typing/change-এর সময় বারবার fire হতে পারে

change
→ value change/commit হওয়ার পর fire হয়
```

NEXT CHAPTER **Chapter 17 Preview — Forms & Form Validation**

পরবর্তী অধ্যায়ে আমরা **Chapter 17 — Forms & Form Validation**-এ যাব। সেখানে HTML forms-এর সাথে JavaScript ব্যবহার করে **input validation, required fields, email/password validation, regex validation, error messages, FormData, submit handling** এবং **professional form validation structure** বিস্তারিত শিখব।